PalmSource™ 99

Using the New Serial Manager

Bruce Thompson
Senior Developer Support Engineer

3Com
More connected.

Using the New Serial Manager

PalmSource 99

- Topics:
  - Existing Serial Manager
  - The New Serial Manager

Existing Serial Manager

- Exists as shared library
  - Library named “Serial Library”
  - Port is referenced by library refNum
  - Library code talks directly to HW
- Functions exist to:
  - Open/Close
  - Read/Write
  - Get Status; Set/Get Settings
  - Etc…

Existing Serial Manager

- Problems:
  - No support for multiple ports
    - Or would require large duplication of code
  - No support for “listening” ports
    - Ports that scan for incoming data and notify client

The New Serial Manager

PalmSource 99

- Why create a New Serial Mgr?
  - Add support for multiple serial devices
  - Consolidate common code into ROM
  - Remove direct HW access by Serial Manager
  - Added extensible model for referencing serial devices
  - Create background ports for monitoring incoming packets

The New Serial Manager

PalmSource 99

- What is changing:
  - Serial Manager is in ROM
  - Physical serial HW as well as virtual devices can be accessed under common API
  - Multiple devices (if available on unit) can be opened and managed at the same time
  - New APIs for getting port info, receiving data, and accessing the RcvQ

The New Serial Manager

PalmSource 99

- Application interaction:
  - The existing model:
    - Ports are opened by opening the “Serial Library” and referred to by the library refNum
  - The New Serial Mgr:
    - Ports are opened by specifying a logical port reference or port creator type
    - Open ports are accessed through a unique portID
    - Applications can select the user preferred port by getting the current Connection Panel profile

The New Serial Manager

- Compatibility
  - Existing Serial Library APIs will continue to be available through a compatibility library
    - Library called “Serial Library”
    - Converts all functions to new API
    - Provides access to the 68328 UART/Cradle serial port only

The New Serial Manager

- Testing for Existence:
  - The existing Serial Mgr:
    - Call SysLibFind with “Serial Library” parameter and see if error was returned
  - The New Serial Mgr:
    - Feature Mgr token: sysFileCSerialMgr (‘smgr’)
      - Err = FtrGet(sysFileCSerialMgr , sysFtrNewSerialPresent, &value);

New Serial Manager Block Diagram



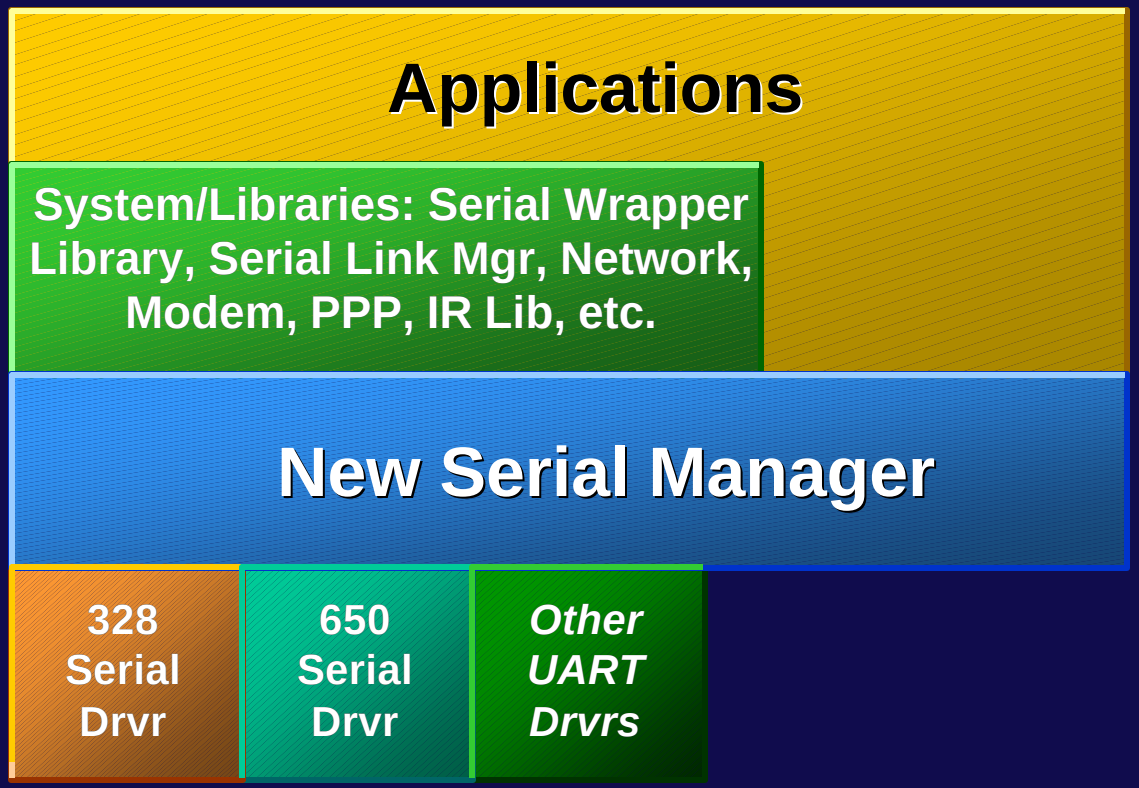
Applications Are Clients of Other Parts of the System, Other Libraries, or the New Serial Manager Directly

New Serial Manager Block Diagram



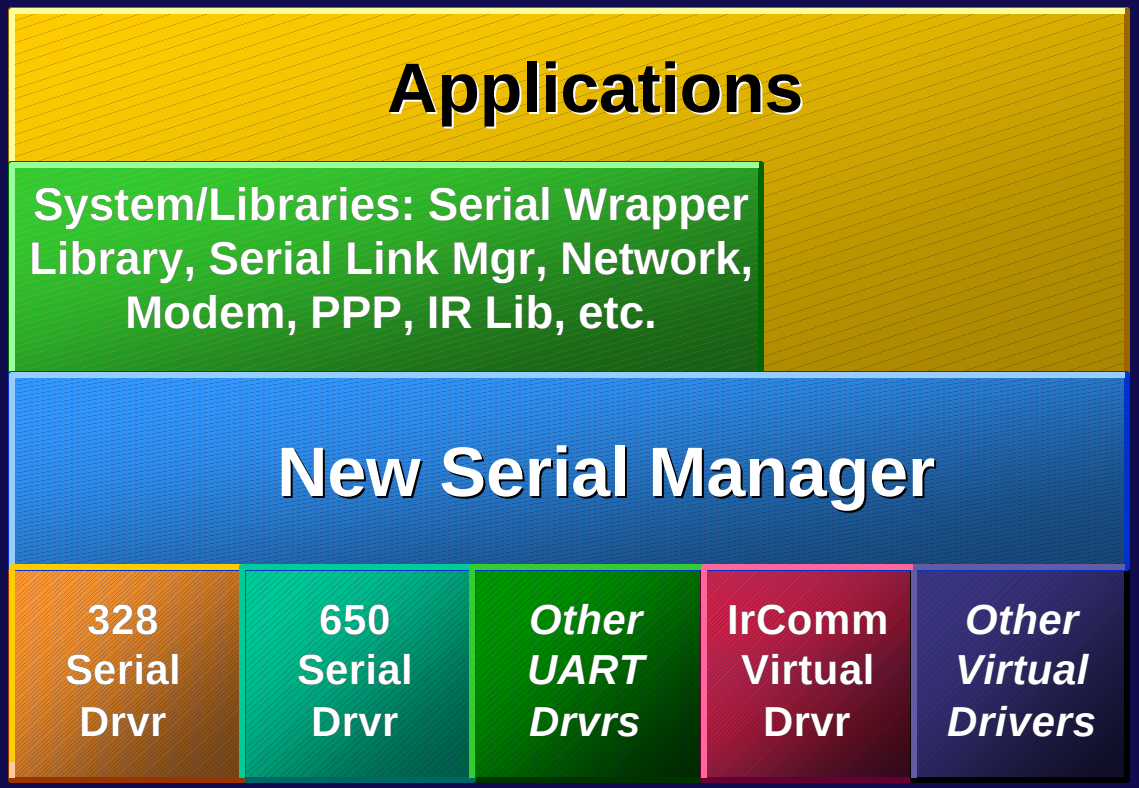
The New Serial Mgr Manages Connections Between all Clients and Open Serial Ports

New Serial Manager Block Diagram



Serial Drvrs Are Small Abstraction Layers to the Physical Hardware which Reside as Prc Databases and Are of Type 'sdrv'

New Serial Manager Block Diagram



Virtual Drvrs Are Like Serial Drvrs Except They Do Not Control Physical Hardware and Abstract Some Other Serial-Like Interface. They Are of Type ‘vdrv’.

SerialMgrNew Features

PalmSource 99

- Getting Info on available ports
  - SrmGetDeviceCount returns number of physical and virtual devices installed
  - SrmGetDeviceInfo returns information about a specific port
    - Port type
    - Port name
    - Port features
    - And more…

SerialMgrNew Features

PalmSource 99

- Opening a port
  - Apps can get the current Connection Mgr profile which contains current port information
  - Or ports can be specified via a logical port reference or a 4-character port creator type
    - serPortCradlePort = $8000
    - serPortIrPort = $8001
    - 68328 port = ‘u328’
    - Ir Comm = ‘ircm’

SerialMgrNew Features

- Getting the portID:
  - Err = SrmOpen(serPortCradlePort, 57600, &portID);
  - Used to reference the open serial port in all New Serial Mgr functions (instead of library refNum)

SerialMgrNew Features

- Background ports:
  - Special open ports that are only able to receive data and notify a client application that data is available
    - Primarily used by IR Library
    - Keyboards may also be good devices…
  - Have control of port only while there is no foreground connection
  - Use SrmOpenBackground function to create (also returns a portID)

SerialMgrNew APIs

- Wakeup Handlers
  - Allow clients to be notified via a callback that data has been received by the serial interrupt routine
    - SrmSetWakeupHandler
    - SrmPrimeWakeupHandler
      - Wakeup handler function called after some number of bytes are received by the interrupt handler

SerialMgrNew APIs

- Wakeup Handlers are called at interrupt time!
  - No Palm OS™ function calls (with a few exceptions)
    - Only OS functions that do not move memory and functions that do not block
  - Generally, applications should set a global variable and call EvtWakeup() to wakeup the UI task and get it to handle events

SerialMgrNew APIs

PalmSource 99

- Fast access to Serial Rcv Q:
  - SrmReceiveWindowOpen
    - Opens direct access to the serial managers rcv Q
    - Provides fast access to data
  - SrmReceiveWindowClose
    - Tells SerialMgr that some number of bytes have been removed from the Q

SerialMgrNew APIs

- Thou Shalt Not!!!
  - Use SrmReceive, SrmReceiveWait, SrmReceiveCheck, or SrmReceiveFlush while a window is open
  - Leave window open for long time
    - This locks down a section of the Q

SerialMgrNew APIs

- Other various changes:
  - SrmGetStatus
    - Returns several HW states back to the application
      - CTS, RTS, and DSR states
      - Break signal state
      - Line errors (returned as parameter)

SerialMgrNew APIs

- Baud rate, port settings flags, and ctsTimeout accessed through SrmControl function opCodes:
  - srmCtlSetBaudRate and srmCtlGetBaudRate
  - srmCtlSetFlags and srmCtlGetFlags
  - srmCtlSetCtsTimeout and srmCtlGetCtsTimeout

SerialMgrNew APIs

- Max Baud Rate and port HW handshaking threshold are accessed through SrmGetDeviceInfo function
- New SrmControl opCode for getting the optimal buffer size for transmitting data
  - srmCtlGetOptimalTransmitSize

SerialMgrNew APIs

PalmSource 99

- All functions begin with ‘Srm’ instead of ‘Ser’
- SrmGetStatus returns line errors in a parameter instead of return value
- SrmSendWait has no timeout value
  - ctsTimeout is used

SerialMgrNew APIs

PalmSource 99

- All parameters that specify buffer sizes are now 32-bits
  - SrmSend and SrmSendCheck
  - SrmReceive, SrmReceiveWait, and SrmReceiveCheck
  - SrmReceiveWindowOpen and SrmReceiveWindowClose
- With one Exception:
  - SrmPrimeWakeupHandler

SerialMgrNew Programming

PalmSource 99

- Sending Data
  - Try not to send extremely large blocks of data nor repeatedly send very small packets of data
  - Find and use the optimal buffer size for a device, especially when using virtual devices
    - srmCtlGetOptimalTransmitSize using SrmControl function
      - May return an error, zero, or positive number

SerialMgrNew Programming

PalmSource 99

- Receiving Data
  - Do not loop on SrmReceiveWait or SrmReceiveCheck without giving other tasks time to handle their events
  - Use Wakeup Handlers and Receive Windows only if needed for speed

The New Serial Manager

PalmSource 99

- When this will be available:
  - New Serial Mgr and serial device model has been released as part of the 3.3 ROM upgrade, available now
    - Use FtrGet to determine if it's available
  - SDK and further documentation is available on the 3.5 seeding CD here at PalmSource™

Conclusion

PalmSource 99

- Read SDK documentation
- Full Compatibility for existing Serial Mgr API via the Serial compatibility library
- This new serial model will be the future for Palm™ serial communications—prepare for its release and new features

PalmSource™ 99

Creating New Serial Drivers for Palm OS™

Bruce Thompson
Senior Developer Support Engineer

3Com
More connected.

Creating New Palm OS™ Serial Drivers

PalmSource 99

- Topics:
  - Serial Drivers (Physical and Virtual)

Creating Serial Drivers

PalmSource 99

- Physical and Virtual Devices
  - “sdrv” or Serial Drvr for physical serial HW only
    - HAL for serial UART HW
    - Small Code resource of type ‘sdrv’
    - Supports basic serial functionality through a specific API
    - Handles all Interrupt Services for HW

Creating Serial Drivers

PalmSource 99

- "vdrv" or Virtual Drvr for virtual ports only
  - Abstraction layer to other software API
  - Small code resource of type 'vdrv'
  - Executes at the same task priority as the client application
  - Read and write calls are block-based instead of byte-based

Serial Drvrs (sdrv)

PalmSource 99

- Functions to be supported
  - DrvrEntryPoint
  - Open, Close
  - Control, Status
  - ReadChar and WriteChar
    - Read and write one-byte at a time
  - ISP for interrupt handling

Serial Drvrs (sdrv)

- DrvrEntryPoint
  - First function in the code resource
  - Called at two different times:
    - At Install to verify HW exists and return info about serial device
    - At Open time to return a struct of function pointers to the sdrv

Serial Drvrs (sdrv)

PalmSource 99

- Interrupts
  - Patch system a-trap IRQHandler for HW specific IRQ line
  - Install ISP at Open and Remove ISP at Close
  - Only needs to identify interrupt and call SaveDataProc function supplied at Open time
  - Every IRQ line has low-memory global which can act as a refcon for the ISP function

Serial Drvrs (sdrv)

- SdrvOpen
  - Initializes HW
  - Installs ISP (via atrap patch)
  - Allocates global variables
  - Sets initial baud rate
  - Prepares HW to receive data

Serial Drvrs (sdrv)

- SdrvClose
  - Frees globals
  - Removes ISP
  - Places HW in lowest power state
- SdrvControl
  - Supports a variety of HW (and some software) specific functions (See SDK materials)

Serial Drvrs (sdrv)

- SdrvStatus
  - Returns several HW states back to Serial Mgr via 16-bit bitfield
    - CTS, RTS, and DSR states
    - Break signal state
    - TX FIFO Empty, TX FIFO Full

Serial Drvrs (sdrv)

- SdrvReadChar
  - Reads byte from FIFO or interprets line error to serial manager error
  - Should be written in 68K Asm
- SdrvWriteChar
  - Writes one byte to HW register

Virtual Drvrs (vdrv)

- Similar to ‘sdrv’, but doesn’t abstract HW
  - No ISP handling
- Data Transfers are block-based instead of byte-based
- Good Examples:
  - IrComm Driver
  - Simulator serial handling
  - A decoder for a serial port

Virtual Drvrs (vdrv)

- Reading Data
  - Reading data is the result of the vdrv being given data by the software API it is abstracting
  - At Open, vdrv's are passed a pointer to DrvRcvQType; it contains:
    - Abstracted pointer to RcvQ
    - WriteByte and WriteBlock function pointers
    - GetQSpace and GetQSize function pointer

Virtual Drvrs (vdrv)

- Writing Data
  - The result of someone calling the SrmSend function
    - Ptr to block of data is handed off
    - Function executes synchronously

Virtual Drvrs (vdrv)

- Status function
  - Exists as in an sdrv and can return the same status signals, but is only used by SdrvStatus function
    - Useful if emulating a serial protocol that may have SW support for handshaking

Virtual Drvrs (vdrv)

- Different Control OpCodes
  - No HW specific opCodes are present
  - vdrvOpCodeSendBufferedData
    - Virtual devices may buffer smaller blocks of transmit data in order to send one large block. This opCode tells the vdrv to send whatever it has buffered immediately
  - vdrvOpCodeGetMaxRcvBlockSize
    - For optimal performance return the maximum block of data to receive from the RcvQ at any one time

Virtual Drvrs (vdrv)

PalmSource 99

- vdrvOpCodeGetOptTransmitSize
  - Returns the optimal transmit buffer size for the virtual device
- vdrvOpCodeNotifyBytesReadFromQ
  - Tells the virtual device that data has been read from the RcvQ

Creating Serial Drivers

PalmSource 99

- More information:
  Consult the SDK documentation
- 10–15 minutes for Q&A


PalmSource™ 99
3Com
More connected.